

系统开发工具基础第二周实验报告

姓名：刘静雅

学号：24160001055

日期：2026 年 9 月 1 日

仓库地址：

<https://github.com/Llj159-lab/sys-dev-tools-lab>

目录

1	实验环境与目录结构	2
2	课上实验	2
2.1	q05: 后台任务、信号处理与清理	2
2.2	q06: 编辑器辅助功能与静态检查	3
2.3	q07: 调试排序程序并使用测试验证	6
2.4	q08: 性能分析与代码优化	7
3	课后练习	10
3.1	参数与 Glob	10
3.2	使用 ls 查看文件信息	11
3.3	进程替换	11
3.4	marco 与 polo	12
3.5	返回码与标准流	13
3.6	信号与任务控制	15
3.7	wait 与 pidwait	16
3.8	SSH 密钥生成	18
3.9	SSH 配置文件	19
3.10	ssh-copy-id	20
3.11	SSH 本地端口转发	21
3.12	mosh 连接与断网恢复	23
3.13	ssh 的 -N 和 -f 参数	24
4	解题感悟	25

1 实验环境与目录结构

本周实验在 WSL 的 Bash 环境中完成，仓库目录为：

```
~/sys-dev-tools-lab/week02
```

本周实验目录结构如下：

```
week02/  
|-- q05/  
|-- q06/  
|-- q07/  
|-- q08/  
|-- report_assets/  
|   |-- practice/  
|-- report.tex
```

实验过程中分别使用了 Shell 后台任务与信号处理、Python 测试与编辑器辅助功能、调试和单元测试，以及性能分析和代码优化等方法。

2 课上实验

2.1 q05：后台任务、信号处理与清理

实验目的

本题练习 Shell 后台任务的启动与管理、进程号获取、信号发送以及清理脚本的编写。

实验过程

进入 q05 目录：

```
cd ~/sys-dev-tools-lab/week02/q05
```

启动后台任务并保存标准输出和标准错误：

```
bash worker.sh > stdout.log 2> stderr.log &  
jobs -l  
jobs -p
```

获取进程号并发送终止信号：

```
pid=$(jobs -p | head -n 1)  
kill -TERM "$pid"
```

执行清理脚本并查看日志：

```
bash cleanup.sh
cat cleanup.log
```

实验结果

实验成功启动了后台任务，并使用 `jobs` 查看了任务状态和进程号。随后使用 `kill -TERM` 向后台任务发送终止信号，任务停止运行。标准输出、标准错误和清理过程分别保存到相应日志文件中。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ cd ~/sys-dev-tools-lab/week02/q05
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ chmod +x worker.sh
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$
```

图 1: 启动后台任务并查看任务状态

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ jobs -p
8570
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ kill -TERM $(jobs -p)
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ wait
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ cat cleanup.log
CLEAN_EXIT
CLEAN_EXIT
CLEAN_EXIT
CLEAN_EXIT
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ cat stdout.log
cat: stdout.log: No such file or directory
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q05$ cat stderr.log
```

图 2: 终止后台任务并执行清理脚本

分析

本题说明了后台任务不会阻塞当前终端，但仍然属于当前 Shell 的任务管理范围。通过 `jobs` 可以查看任务，通过进程号和 `kill` 可以向进程发送信号。日志重定向和清理脚本提高了实验过程的可复现性。

2.2 q06: 编辑器辅助功能与静态检查

实验目的

本题练习查找引用、跳转到定义、符号重命名、静态检查和自动化测试。

实验过程

进入 q06 目录：

```
cd ~/sys-dev-tools-lab/week02/q06
```

在编辑器中查找函数或变量的引用位置，并从调用位置跳转到定义位置。完成符号重命名后，检查相关文件中的引用是否同步更新。

执行静态检查：

```
ruff check .
```

修正未使用导入等问题后，运行测试：

```
pytest -q
```

实验结果

实验完成了代码引用查找、定义跳转和符号重命名。静态检查发现了未使用的导入问题，修改后重新检查，代码检查结果正常。最后运行 pytest，测试全部通过。

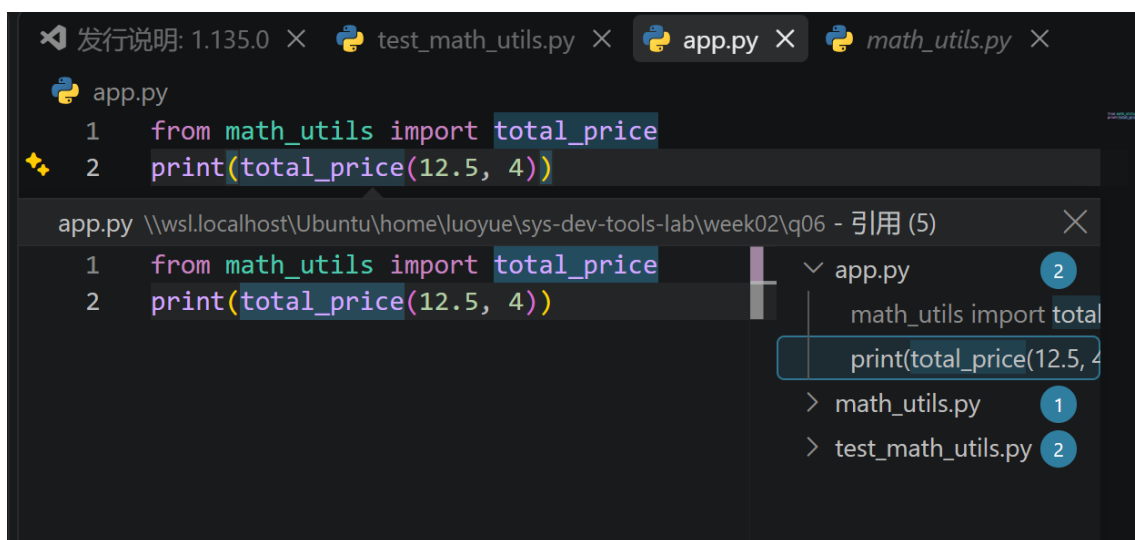


图 3: 查找符号引用

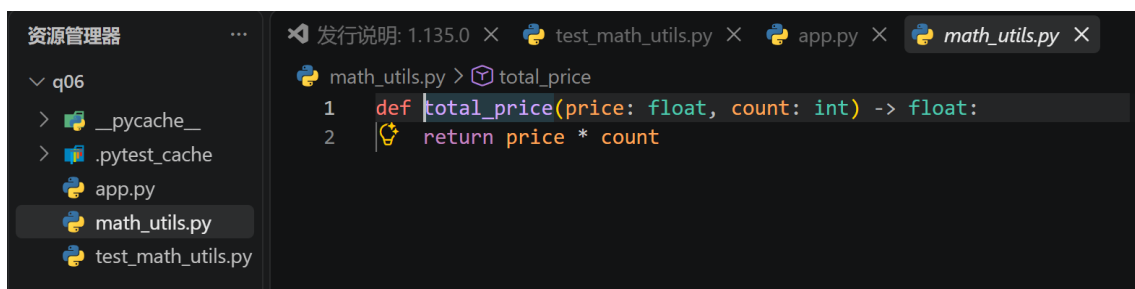


图 4: 跳转到符号定义

```

./pytest_cache/v/cache/nodeids:2: "test_math_utils.py::test_total_price"
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ grep -R -n --exclude-dir=__pycache__ \
> -E 'total_price|calculate_total' .
./math_utils.py:1:def calculate_total(price: float, count: int) -> float:
./app.py:1:from math_utils import calculate_total
./app.py:2:print(calculate_total(12.5, 4))
./test_math_utils.py:1:from math_utils import calculate_total
./test_math_utils.py:2:def test_total_price():
./test_math_utils.py:3:    assert calculate_total(12.5, 4) == 50.0
./pytest_cache/v/cache/nodeids:2: "test_math_utils.py::test_total_price"

```

图 5: 重命名符号并同步修改引用

```

(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ ruff check .
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1 | / import os
2 | | from math_utils import calculate_total
3 | |
4 | | print(calculate_total(12.5, 4))
help: Organize imports
1 | import os
2 | +
3 | from math_utils import calculate_total

F401 [*] `os` imported but unused
--> app.py:1:8
1 | import os
2 |
3 | from math_utils import calculate_total
help: Remove unused import: `os`
- import os
1 | from math_utils import calculate_total

I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
1 | from math_utils import calculate_total
2 | def test_total_price():
3 |     assert calculate_total(12.5, 4) == 50.0
help: Organize imports
1 | from math_utils import calculate_total
2 | +
3 | +
4 | def test_total_price():

Found 3 errors.
[*] 3 fixable with the `--fix` option.

```

图 6: 静态检查发现未使用的导入

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ ruff format .
3 files reformatted
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ ruff check .
All checks passed!
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ pytest -q
. [100%]
1 passed in 0.01s
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$ python app.py
50.0
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q06$
```

图 7: 修改后重新检查并运行测试

分析

编辑器辅助功能可以提高代码阅读和修改效率。查找引用有助于确定修改范围，跳转定义有助于理解函数实现，符号重命名可以减少手工修改造成的遗漏。静态检查和自动化测试分别从代码质量和功能正确性两个方面验证了修改结果。

2.3 q07: 调试排序程序并使用测试验证

实验目的

本题要求运行排序程序和测试代码，根据测试失败信息定位错误，修改程序缺陷，并重新运行测试。

实验过程

进入 q07 目录：

```
cd ~/sys-dev-tools-lab/week02/q07
pytest -q
```

根据测试输出定位错误位置，检查归并排序中左右子数组的下标访问、循环条件和剩余元素复制过程。

修改程序后重新运行：

```
pytest -q
```

实验结果

初次运行测试时，排序程序出现失败。通过查看测试信息和分析程序执行过程，定位到归并排序中的下标访问错误。修改后重新运行 pytest，所有测试均通过。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ nano merge_sort.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ python merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 8: 修复前的测试失败结果

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ nano merge_sort.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ python merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
```

图 9: 修改排序程序后的状态

```
[1, 1, 2, 3, 4, 5, 6, 9]
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ nano test_merge_sort.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q07$ pytest -q
..
2 passed in 0.01s
```

图 10: 修复后测试全部通过

分析

本题体现了测试驱动调试的基本过程：先通过测试发现问题，再根据错误信息定位程序位置，分析控制流程和数组边界，最后重新运行测试验证修复结果。

2.4 q08: 性能分析与代码优化

实验目的

本题要求生成测试数据，运行优化前后的单词频率统计程序，使用 `time` 测量运行时间，并使用 `cProfile` 定位主要耗时部分。

实验过程

进入 q08 目录并生成数据：

```
cd ~/sys-dev-tools-lab/week02/q08
python generate_words.py
wc -w words.txt
sha256sum words.txt
```

运行优化前程序：

```
/usr/bin/time -f '%e' -o before1.time \
python wordfreq_slow.py > before1.out

/usr/bin/time -f '%e' -o before2.time \
python wordfreq_slow.py > before2.out
```

使用 `cProfile` 分析：

```
python -m cProfile -s cumulative wordfreq_slow.py \
| tee profile_before.txt
```

运行优化后的程序：

```
/usr/bin/time -f '%e' -o after1.time \  
python wordfreq.py > after1.out  
  
/usr/bin/time -f '%e' -o after2.time \  
python wordfreq.py > after2.out
```

比较输出：

```
diff -u before1.out after1.out  
diff -u before2.out after2.out
```

实验结果

如下图所示，words.txt 中共有 30000 个单词。优化前两次运行时间均为 0.21 s，优化后两次运行时间均为 0.02 s。

cProfile 结果显示，优化前程序共有 1012 次函数调用，累计耗时为 0.185 s，其中 wordfreq_slow.py 主流程累计耗时为 0.183 s。

优化前后的程序输出均为：

```
count=1000
```

因此，优化过程没有改变程序的输出结果。

两次运行时间的中位数及加速比为：

- 优化前中位数：0.210000 s；
- 优化后中位数：0.020000 s；
- 性能提升：10.50 倍。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ nano generate_words.py  
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ python generate_words.py  
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ ls -lh words.txt  
-rw-r--r-- 1 luoyue luoyue 144K Aug 31 15:20 words.txt  
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ wc -w words.txt  
30000 words.txt  
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ sha256sum words.txt  
9e47c07f20ee78ae80ad031c4cd1013ff644ea9517f5d16d5002ec637fbfba14 words.txt  
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$
```

图 11: 生成 30000 个测试单词

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ nano wordfreq.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cp wordfreq.py wordfreq_slow.py
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ /usr/bin/time -f '%e' -o before1.time \
python wordfreq_slow.py > before1.out

/usr/bin/time -f '%e' -o before2.time \
python wordfreq_slow.py > before2.out
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat before1.out
count= 1000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat before2.out
count= 1000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat before1.time
0.21
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat before2.time
0.21
```

图 12: 优化前两次运行结果和时间

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ python -m cProfile -s cumulative wordfreq_slow.py
| tee profile_before.txt
count= 1000
1012 function calls in 0.185 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.185    0.185 {built-in method builtins.exec}
1      0.183    0.183    0.185    0.185 wordfreq_slow.py:1(<module>)
1      0.002    0.002    0.002    0.002 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {method 'read' of 'io.TextIOWrapper' objects}
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:319(decode)
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 {built-in method io.open}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1      0.000    0.000    0.000    0.000 <frozen codecs>:309(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
1      0.000    0.000    0.000    0.000 <frozen codecs>:260(__init__)

(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ grep -nE 'ncalls|wordfreq_slow|cumtime|tottime' p
rofile_before.txt
6:  ncalls  tottime  percall  cumtime  percall filename:lineno(function)
8:  1      0.183    0.183    0.185    0.185 wordfreq_slow.py:1(<module>)
```

图 13: 优化前的 cProfile 分析结果

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ sha256sum words.txt
9e47c07f20ee78ae80ad031c4cd1013ff644ea9517f5d16d5002ec637fbfba14 words.txt
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ /usr/bin/time -f '%e' -o after1.time \
> python wordfreq.py > after1.out
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ /usr/bin/time -f '%e' -o after2.time \
> python wordfreq.py > after2.out
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat after1.out
count= 1000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat after2.out
count= 1000
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat after1.time
0.02
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ cat after2.time
0.02
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/q08$ diff -u before1.out after1.out
```

图 14: 优化后两次运行结果和时间

```
(.venv) luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/q00$ before_median=$(awk '{sum += $1} END {printf "%.6f\n", sum/NR}' before1.time before2.time)
(.venv) luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/q00$ echo "$before_median"
0.210000
(.venv) luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/q00$ after_median=$(awk '{sum += $1} END {printf "%.6f\n", sum/NR}' after1.time after2.time)
(.venv) luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/q00$ echo "$after_median"
0.020000
(.venv) luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/q00$ awk -v before="$before_median" -v after="$after_median" 'BEGIN { printf "%.2fx\n", before/after }'
10.50x
```

图 15: 优化前后运行时间中位数及加速比

分析

优化前程序两次运行均需要 0.21 s，优化后均只需要 0.02 s。在测试数据和程序输出保持一致的情况下，运行时间减少了约 90.5%，整体性能提升约 10.50 倍。

本题采用了建立性能基准、使用分析工具定位瓶颈、修改程序并验证输出的流程。实验表明，性能优化必须建立在实际测量基础上，同时还要确认优化没有改变程序功能。

3 课后练习

3.1 参数与 Glob

题目

练习命令参数、特殊参数和 Glob 模式，观察不同模式对文件名的匹配效果。

操作步骤

```
cd ~/sys-dev-tools-lab/week02/practice
mkdir -p q01
cd q01

touch a.txt b.txt c.txt file1.txt file2.txt file10.txt notes.md

ls *.txt
ls file?.txt
ls {a,b,c}.txt
```

分析

*.txt 可以匹配任意长度的文件名, file?.txt 中的问号匹配一个字符, {a,b,c}.txt 会展开为三个具体文件名。Glob 和花括号展开都发生在 Shell 执行命令之前，但二者的语义不同。

```

luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab$ mkdir -p week02/practice/q01
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab$ cd week02/practice/q01
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q01$ touch a.txt b.txt c.txt file1.txt file2.txt file
10.txt notes.md
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q01$ ls *.txt
a.txt b.txt c.txt file1.txt file10.txt file2.txt
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q01$ ls file?.txt
file1.txt file2.txt
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q01$ ls {a,b,c}.txt
a.txt b.txt c.txt

```

图 16: Glob 模式匹配结果

3.2 使用 ls 查看文件信息

题目

阅读 man ls，写出一个命令，使其包含隐藏文件、以易读格式显示文件大小、按最近修改时间排序并显示颜色。

操作步骤

```

cd ~/sys-dev-tools-lab/week02/practice
ls -lahtr --color=auto

```

分析

-a 显示隐藏文件，-h 以易读格式显示文件大小，-t 按修改时间排序，-r 反向排序，--color=auto 根据终端环境显示颜色。

```

luoyue@LAPTOP-20PGDGAI:~/week02/practice/q02$ ls -l / | head
total 2772
drwxr-xr-x  3 root root    4096 Aug  4 09:12 Docker
lrwxrwxrwx  1 root root      7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x  2 root root    4096 Feb 26  2024 bin.usr-is-merged
drwxr-xr-x  2 root root    4096 Apr 22  2024 boot
drwxr-xr-x 15 root root   3860 Aug 31 13:52 dev
drwxr-xr-x 102 root root   4096 Aug 31 23:48 etc
-rw-r--r--  1 root root     32 Nov 14  2025 flag
drwxr-xr-x  3 root root    4096 Nov 14  2025 home
-rwxrwxrwx  1 root root 2735264 Aug  7  2025 init

```

图 17: ls 命令参数组合结果

3.3 进程替换

题目

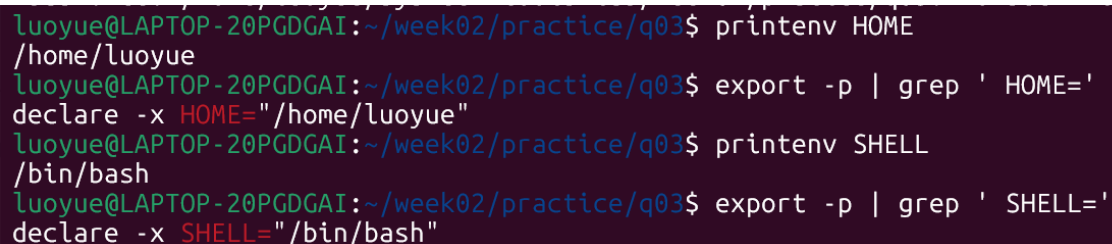
练习进程替换，使用 diff 比较两个命令输出，并分析 printenv 与 export 输出不同的原因。

操作步骤

```
cd ~/sys-dev-tools-lab/week02/practice
diff <(printenv | sort) <(export -p | sort)
```

分析

<(command) 会把命令输出提供为类似文件的输入，使 diff 可以直接比较两个命令的输出。由于 printenv 和 export -p 的输出格式和内容范围不同，因此比较结果可能存在差异。



```
luoyue@LAPTOP-20PGDGAI:~/week02/practice/q03$ printenv HOME
/home/luoyue
luoyue@LAPTOP-20PGDGAI:~/week02/practice/q03$ export -p | grep 'HOME='
declare -x HOME="/home/luoyue"
luoyue@LAPTOP-20PGDGAI:~/week02/practice/q03$ printenv SHELL
/bin/bash
luoyue@LAPTOP-20PGDGAI:~/week02/practice/q03$ export -p | grep 'SHELL='
declare -x SHELL="/bin/bash"
```

图 18: 进程替换和 diff 比较结果

3.4 marco 与 polo

题目

编写两个 Bash 函数 marco 和 polo。执行 marco 时保存当前工作目录，执行 polo 时返回该目录。

操作步骤

```
cd ~/sys-dev-tools-lab/week02/practice
mkdir -p q04
cd q04

cat > marco.sh <<'EOF'
marco() {
    export MARCO_PWD="$PWD"
}

polo() {
    cd "$MARCO_PWD"
}
EOF
```

```
source marco.sh
```

```
cd /tmp
```

```
marco
```

```
cd /
```

```
polo
```

```
pwd
```

分析

marco 将当前目录保存到环境变量，polo 读取该变量并执行 cd。必须使用 source 将函数加载到当前 Shell 中，否则函数无法改变当前终端的工作目录。

```
luoyue@LAPTOP-20PGDGAI:~/week02/practice$ cat > marco.sh <<'EOF'
> marco() {
    export MARCO_PWD="$PWD"
}

polo() {
    cd "$MARCO_PWD"
}
EOF
luoyue@LAPTOP-20PGDGAI:~/week02/practice$ source marco.sh
luoyue@LAPTOP-20PGDGAI:~/week02/practice$ cd /tmp
luoyue@LAPTOP-20PGDGAI:/tmp$ marco
luoyue@LAPTOP-20PGDGAI:/tmp$ cd /
luoyue@LAPTOP-20PGDGAI:/$ polo
luoyue@LAPTOP-20PGDGAI:/tmp$ pwd
/tmp
luoyue@LAPTOP-20PGDGAI:/tmp$
```

图 19: marco 和 polo 函数测试结果

3.5 返回码与标准流

题目

编写一个可能偶尔失败的 Bash 脚本，将标准输出和标准错误分别保存到文件，并记录脚本失败前运行的次数。

操作步骤

```
cd ~/sys-dev-tools-lab/week02/practice
```

```
cat > debug.sh <<'EOF'
#!/usr/bin/env bash

n=$(( RANDOM % 100 ))

if [[ $n -eq 42 ]]; then
    echo "Something went wrong"
    echo "The error was using magic numbers" >&2
    exit 1
fi

echo "Everything went according to plan"
EOF

chmod +x debug.sh

count=0
while true; do
    count=$((count + 1))
    ./debug.sh > stdout.txt 2> stderr.txt || break
done

echo "failed after $count runs"
cat stdout.txt
cat stderr.txt
```

分析

脚本成功时返回码为 0，失败时通过 `exit 1` 返回非零状态。标准输出重定向到 `stdout.txt`，标准错误重定向到 `stderr.txt`。循环中的 `break` 在脚本失败后结束循环。

```

luoyue@LAPTOP-20PGDGAI:/tmp$ cat > debug.sh <<'EOF'
> #!/usr/bin/env bash

n=$(( RANDOM % 100 ))
if [[ $n -eq 42 ]]; then
    echo "Something went wrong"
    echo "The error was using magic numbers" >&2
    exit 1
fi

echo "Everything went according to plan"
EOF
luoyue@LAPTOP-20PGDGAI:/tmp$ chmod +x debug.sh
luoyue@LAPTOP-20PGDGAI:/tmp$ count=0
luoyue@LAPTOP-20PGDGAI:/tmp$ while true; do
>   count=$((count + 1))
>   ./debug.sh >stdout.txt 2>stderr.txt || break
> done
luoyue@LAPTOP-20PGDGAI:/tmp$ echo "failed after $count runs"
failed after 48 runs
luoyue@LAPTOP-20PGDGAI:/tmp$ cat stdout.txt
Something went wrong
luoyue@LAPTOP-20PGDGAI:/tmp$ cat stderr.txt
The error was using magic numbers

```

图 20: 返回码、标准输出和标准错误结果

3.6 信号与任务控制

题目

在终端里启动一个 `sleep 10000` 任务，用 `Ctrl-Z` 把它挂起，再用 `bg` 让它继续在后台运行。然后使用 `pgrep` 找到它的 PID，并使用 `pkill` 将它终止，整个过程不要手动输入 PID。

操作步骤

```

cd ~/sys-dev-tools-lab/week02/practice/q06

sleep 10000

```

按下 `Ctrl-Z`，然后执行：

```

bg
jobs -l
pgrep -af 'sleep 10000'
pkill -f 'sleep 10000'
pgrep -af 'sleep 10000' || echo "sleep 10000 has been killed"

```

```
jobs
```

分析

Ctrl-Z 会暂停前台任务，bg 会使暂停任务在后台继续运行。pgrep -af 按完整命令行查找进程，kill -f 按完整命令行匹配并终止进程，因此不需要手工输入 PID。

```
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab$ cd ~/sys-dev-tools-lab/week02/practice/q06
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/practice/q06$ sleep 10000
^Z
[1]+  Stopped                  sleep 10000
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/practice/q06$ bg
[1]+  sleep 10000 &
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/practice/q06$ jobs -l
-bash: jobs: -l: invalid option
jobs: usage: jobs [-lnprs] [jobspec ...] or jobs -x command [args]
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/practice/q06$ pgrep -af 'sleep 10000'
23964 sleep 10000
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/practice/q06$ kill -f 'sleep 10000'
[1]+  Terminated              sleep 10000
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/practice/q06$ pgrep -af 'sleep 10000' || echo "sleep 10000 has
been killed"
sleep 10000 has been killed
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week02/practice/q06$ jobs
```

图 21: 后台任务、进程查找和进程终止

3.7 wait 与 pidwait

题目

启动一个后台进程，等待它结束后再执行其他命令。然后编写一个 pidwait Bash 函数，接收一个 PID，并一直等待该进程结束。

操作步骤

```
cd ~/sys-dev-tools-lab/week02/practice/q06

sleep 5 &
pid=$!
echo "started pid=$pid"

wait "$pid"
echo "sleep finished"
ls
```

编写 pidwait 函数：

```
cat > pidwait.sh <<'EOF'
#!/usr/bin/env bash

pidwait() {
```

```

local pid="$1"

while kill -0 "$pid" 2>/dev/null; do
    sleep 1
done
}
EOF

source pidwait.sh

sleep 5 &
pid=$!

echo "waiting for pid=$pid"
pidwait "$pid"
echo "pid finished"

```

分析

\$! 表示最近启动的后台进程的 PID，wait 等待当前 Shell 启动的后台子进程结束。kill -0 不会真正终止进程，只用于检测进程是否存在。循环中的 sleep 1 可以避免持续占用 CPU。

```

luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ sleep 5 &
[1] 24055
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ pid=$!
[1]+  Done                  sleep 5
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ echo "started pid=$pid"
started pid=24055
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ wait "$pid"
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ echo "sleep finished, now run ls"
sleep finished, now run ls
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ ls
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ nano pidwait.sh
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ source pidwait.sh
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ sleep 5 &
[1] 24111
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ pid=$!
[1]+  Done                  sleep 5
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ echo "waiting for pid=$pid"
waiting for pid=24111
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ pidwait "$pid"
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q0c$ echo "pid finished"
pid finished

```

图 22: wait 和 pidwait 的运行结果

3.8 SSH 密钥生成

题目

进入 `~/.ssh/`, 检查是否已经存在 SSH 密钥。如果没有, 使用 `ssh-keygen -a 100 -t ed25519` 生成密钥。

操作步骤

```
cd ~
ls -la ~/.ssh
ssh-keygen -a 100 -t ed25519
ls -la ~/.ssh
```

分析

`id_ed25519` 是私钥, 不能泄露; `id_ed25519.pub` 是公钥, 可以复制到远程服务器。SSH 公钥认证可以减少密码登录, 并提高远程登录的安全性。

```
luoyue@LAPTOP-20PGDGAI:~$ ls -la ~/.ssh
ls: cannot access '/home/luoyue/.ssh': No such file or directory
luoyue@LAPTOP-20PGDGAI:~$ ssh-keygen -a 100 -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/luoyue/.ssh/id_ed25519):
Created directory '/home/luoyue/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/luoyue/.ssh/id_ed25519
Your public key has been saved in /home/luoyue/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:YLr7/Rhn8ivfYiKSc2VkJQ9TSA1zKkRproWYcP6WQcdI luoyue@LAPTOP-20PGDGAI
The key's randomart image is:
+--[ED25519 256]--+
|    o+.o+*.      |
|   +oE.*o+       |
|  . =oOo. .      |
|  =oB.+          |
| o.. +S.         |
|   . o           |
| .. oo o         |
| +.o.oBo.        |
| .=.o*=o.        |
+----[SHA256]-----+
luoyue@LAPTOP-20PGDGAI:~$ ls -la ~/.ssh
total 16
drwx----- 2 luoyue luoyue 4096 Sep  1 13:55 .
drwxr-x--- 20 luoyue luoyue 4096 Sep  1 13:55 ..
-rw----- 1 luoyue luoyue  419 Sep  1 13:55 id_ed25519
-rw-r--r-- 1 luoyue luoyue  104 Sep  1 13:55 id_ed25519.pub
```

图 23: 检查并生成 SSH 密钥

3.9 SSH 配置文件

题目

编辑 `~/.ssh/config`，配置远程虚拟机别名、用户名、IP 地址、私钥路径和端口转发规则。

操作步骤

```
cd ~  
nano ~/.ssh/config
```

配置内容：

```
Host vm  
    User username_goes_here  
    HostName ip_goes_here  
    IdentityFile ~/.ssh/id_ed25519  
    LocalForward 9999 localhost:8888
```

然后执行：

```
chmod 600 ~/.ssh/config  
ssh -G vm | head
```

分析

配置主机别名后，可以使用 `ssh vm` 连接远程机器。`LocalForward` 为后续本地端口转发实验提供配置，权限设置可以避免其他用户读取 SSH 配置。

```
luoyue@LAPTOP-20PGDGAI:~$ cd ~
luoyue@LAPTOP-20PGDGAI:~$ nano ~/.ssh/config
luoyue@LAPTOP-20PGDGAI:~$ chmod 600 ~/.ssh/config
luoyue@LAPTOP-20PGDGAI:~$ ssh -G vm | head
host vm
user luoyue
hostname 172.25.85.135
port 22
addressfamily any
batchmode no
canonicalizefallbacklocal yes
canonicalizehostname false
checkhostip no
compression no
luoyue@LAPTOP-20PGDGAI:~$
```

图 24: SSH 配置文件及配置检查

3.10 ssh-copy-id

题目

使用 `ssh-copy-id vm` 将 SSH 公钥复制到远程服务器，并测试密钥登录。

操作步骤

```
cd ~
ssh-copy-id vm
ssh vm
whoami
hostname
exit
```

分析

`ssh-copy-id` 会将本机公钥写入远程服务器的 `authorized_keys` 文件。复制成功后，SSH 客户端可以使用本机私钥完成身份验证。

```

luoyue@LAPTOP-20PGDGA1:~$ ssh-copy-id vm
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/luoyue/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
luoyue@172.25.85.135's password:
bind [127.0.0.1]:9999: Address already in use
channel_setup_fwd_listener_tcpip: cannot listen to port: 9999
Could not request local forwarding.

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'vm'"
and check to make sure that only the key(s) you wanted were added.

luoyue@LAPTOP-20PGDGA1:~$ ssh vm
bind [127.0.0.1]:9999: Address already in use
channel_setup_fwd_listener_tcpip: cannot listen to port: 9999
Could not request local forwarding.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep  1 14:21:26 CST 2026

System load:  0.05               Processes:    47
Usage of /:   0.6% of 1006.85GB   Users logged in: 1
Memory usage: 7%                IPv4 address for eth0: 172.25.85.135
Swap usage:  0%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop
Last login: Tue Sep  1 14:21:42 2026 from 172.25.85.135
luoyue@LAPTOP-20PGDGA1:~$ whoami
luoyue
luoyue@LAPTOP-20PGDGA1:~$ hostname
LAPTOP-20PGDGA1
luoyue@LAPTOP-20PGDGA1:~$ exit
logout
Connection to 172.25.85.135 closed.

```

图 25: 复制 SSH 公钥并测试登录

3.11 SSH 本地端口转发

题目

在虚拟机中运行 `python -m http.server 8888`, 然后在本机访问 `http://localhost:9999`, 验证 SSH 本地端口转发。

操作步骤

在远程虚拟机中运行：

```

mkdir -p ~/webtest
cd ~/webtest
echo "hello from vm" > index.html
python -m http.server 8888

```

在本机浏览器中访问：

```
http://localhost:9999
```

分析

SSH 会将本机的 9999 端口转发到远程机器的 8888 端口。因此访问本机的 9999 端口时，实际访问的是远程虚拟机中的 Web 服务。

```
luoyue@LAPTOP-20PGDGAI:~/webtest$ ssh vm
bind [127.0.0.1]:9999: Address already in use
channel_setup_fwd_listener_tcpip: cannot listen to port: 9999
Could not request local forwarding.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep  1 14:30:02 CST 2026

System load:  0.0               Processes:            59
Usage of /:   0.6% of 1006.85GB Users logged in:      1
Memory usage: 7%               IPv4 address for eth0: 172.25.85.135
Swap usage:   0%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop
Last login: Tue Sep  1 14:24:03 2026 from 172.25.85.135
luoyue@LAPTOP-20PGDGAI:~$ mkdir -p ~/webtest
luoyue@LAPTOP-20PGDGAI:~$ cd ~/webtest
luoyue@LAPTOP-20PGDGAI:~/webtest$ echo "hello from vm" > index.html
luoyue@LAPTOP-20PGDGAI:~/webtest$ python -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...
127.0.0.1 - - [01/Sep/2026 14:30:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Sep/2026 14:30:04] code 404, message File not found
127.0.0.1 - - [01/Sep/2026 14:30:04] "GET /favicon.ico HTTP/1.1" 404 -
```

图 26: 远程虚拟机中的 Web 服务

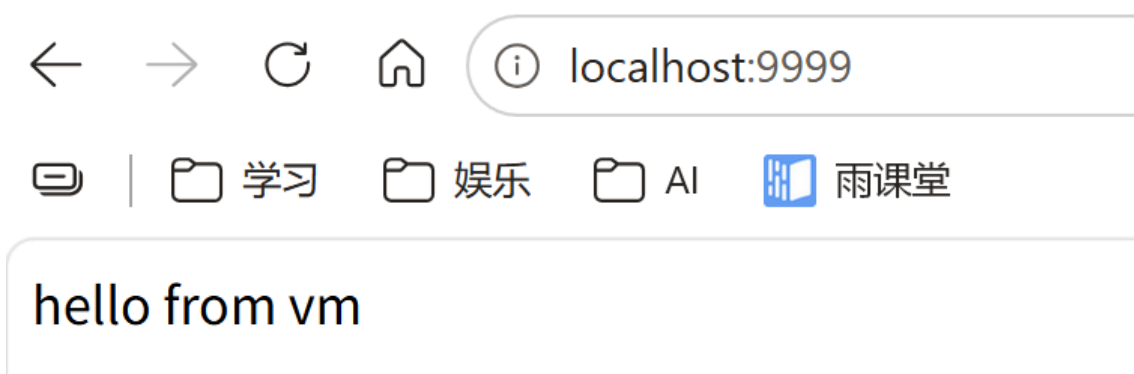


图 27: 本机浏览器访问端口转发服务

3.12 mosh 连接与断网恢复

题目

在虚拟机中安装 mosh 并建立连接，然后断开网络并恢复，观察连接是否能够继续使用。

操作过程

在本机和虚拟机中安装 mosh：

```
sudo apt update
sudo apt install -y mosh
```

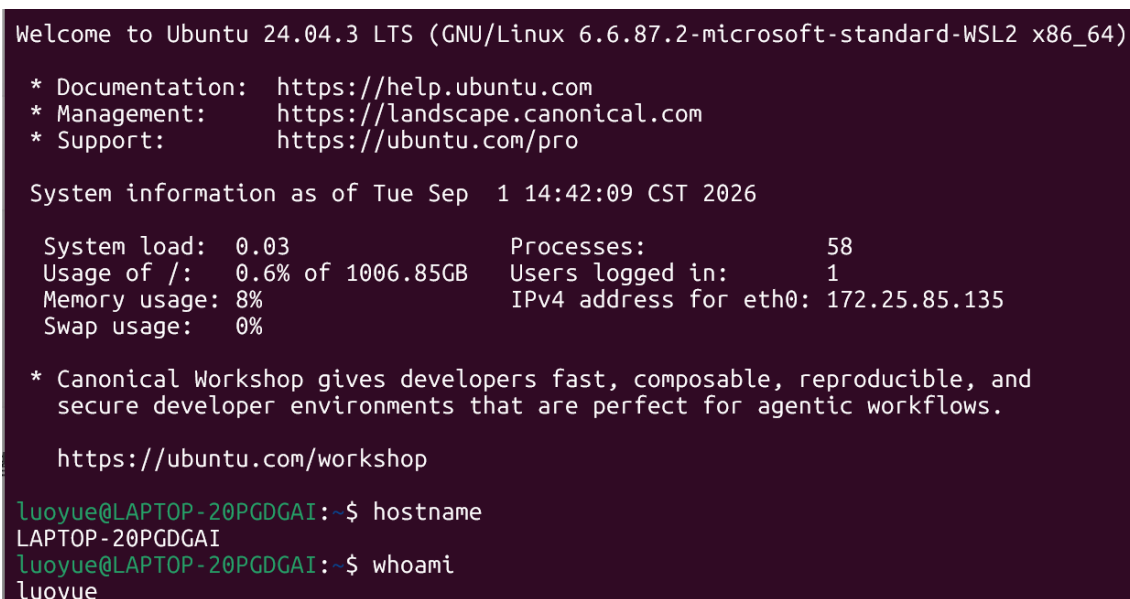
建立连接：

```
mosh vm
hostname
whoami
```

建立连接后暂时断开网络，恢复网络后再次观察终端状态。

实验结果

图 28 为断网前的终端状态，系统显示为 Ubuntu 24.04.3 LTS，主机名为 LAPTOP-20PGDGAI，当前用户为 luoyue，IPv4 地址为 172.25.85.135。随后执行 hostname 和 whoami，分别得到主机名和用户名，说明断网前终端连接正常。



```
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Tue Sep  1 14:42:09 CST 2026

System load:  0.03            Processes:            58
Usage of /:   0.6% of 1006.85GB Users logged in:     1
Memory usage: 8%             IPv4 address for eth0: 172.25.85.135
Swap usage:   0%

* Canonical Workshop gives developers fast, composable, reproducible, and
  secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop

luoyue@LAPTOP-20PGDGAI:~$ hostname
LAPTOP-20PGDGAI
luoyue@LAPTOP-20PGDGAI:~$ whoami
luoyue
```

图 28: 断网前的 mosh 终端状态

图 29 记录了网络恢复后的终端状态，恢复后仍然可以执行 `hostname` 和 `whoami`，输出仍为 `LAPTOP-20PGDGAI` 和 `luoyue`。同时，IPv4 地址仍为 `172.25.85.135`，说明恢复后的环境仍保持一致。

与断网连接前相比，系统进程数从 58 变为 61，系统负载仍为 0.03。这种变化属于系统运行过程中进程数量的正常波动，并不影响主机名、用户名和终端操作结果。

```
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Tue Sep  1 14:45:08 CST 2026

System load:  0.03               Processes:            61
Usage of /:   0.6% of 1006.85GB  Users logged in:     1
Memory usage: 8%                IPv4 address for eth0: 172.25.85.135
Swap usage:   0%

* Canonical Workshop gives developers fast, composable, reproducible, and
  secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop

luoyue@LAPTOP-20PGDGAI:~$ hostname
LAPTOP-20PGDGAI
luoyue@LAPTOP-20PGDGAI:~$ whoami
luoyue
luoyue@LAPTOP-20PGDGAI:~$
```

图 29: 网络恢复后的 mosh 终端状态

分析

如图所示，断网前后使用的是同一台 Ubuntu WSL 环境，主机名、用户名和 IPv4 地址均保持一致，恢复后终端仍然能够执行命令。这说明网络恢复后实验环境能够继续使用。

3.13 ssh 的 -N 和 -f 参数

题目

查询 SSH 命令中的 `-N` 和 `-f` 参数含义，并写出一条能够实现后台端口转发的命令。

操作步骤

查询参数：

```
man ssh
ssh -h 2>&1 | grep -E -- '-N|-f'
```

建立后台端口转发：

```
ssh -N -f -L 9999:localhost:8888 vm
```

查看后台 SSH 进程：

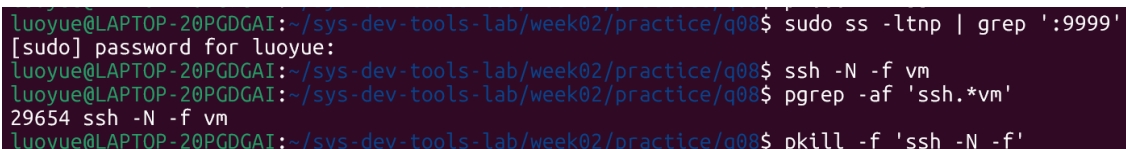
```
pgrep -af 'ssh.*9999'
```

停止端口转发：

```
pkill -f 'ssh.*9999'
```

分析

-N 表示不执行远程命令，只建立 SSH 连接，通常用于端口转发。-f 表示认证完成后将 SSH 进程放入后台运行。二者结合后，可以在不占用当前交互式终端的情况下建立后台端口转发。



```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q08$ sudo ss -ltnp | grep ':9999'
[sudo] password for luoyue:
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q08$ ssh -N -f vm
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q08$ pgrep -af 'ssh.*vm'
29654 ssh -N -f vm
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week02/practice/q08$ pkill -f 'ssh -N -f'
```

图 30: ssh 的 -N 和 -f 参数及后台端口转发

4 解题感悟

通过本周课上实验，掌握了 Shell 后台任务管理、进程信号、标准流重定向以及清理脚本的基本方法。同时，通过 Python 项目的静态检查、单元测试和调试实验，进一步理解了代码导航、错误定位和测试验证在软件开发过程中的作用。q08 的性能实验还说明，程序优化应建立在实际测量和性能分析的基础上，不能仅凭主观判断修改代码。

课后练习进一步扩展了对 Shell 机制和远程开发工具的认识。通过 Glob、进程替换、环境变量、返回码和任务控制等练习，掌握了命令组合和脚本自动化的基本思路；通过 SSH 密钥、配置文件、公钥复制、端口转发、mosh 以及后台 SSH 连接等练习，了解了远程登录和远程服务访问的常用方法。整体实验过程强化了“先观察现象、再分析原因、最后验证结果”的问题解决方法。